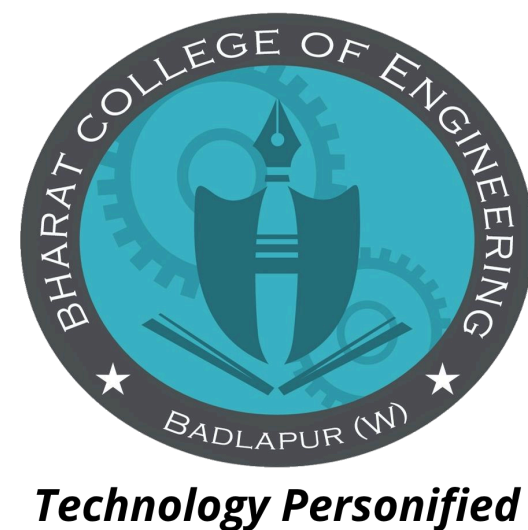




Bharat College of Engineering, Kanhor, Badlapur (W)

MITRA AI Chatbot

“Multifunctional Intelligent Tactful Responsive Assistant”

Team Member's Name: Yash Pal, Ganesh Sharma, Harshad Bhutal, Pranav Patil

Name of the Guide: Prof. Shital Gujar

Project Domain: Artificial Intelligence

Project Group Number: Group number 1

Year: BE

Semester: VIII



CONTENTS

- Abstract
- Introduction
- Problem Statement
- Methodology
- Literature Review
- Design & Experimental Process
- Algorithms Used
- Results
- Conclusion
- References
- Our Research Paper
- Future Scope
- Questionnaire

ABSTRACT

MITRA is a smart, multi-functional virtual assistant designed to provide thoughtful and responsive interaction. This project aims to combine natural language processing, machine learning, and intelligent task handling into one comprehensive chatbot system. The assistant is capable of answering queries, performing tasks, and integrating with other services, ensuring a high-end user experience across multiple domains.

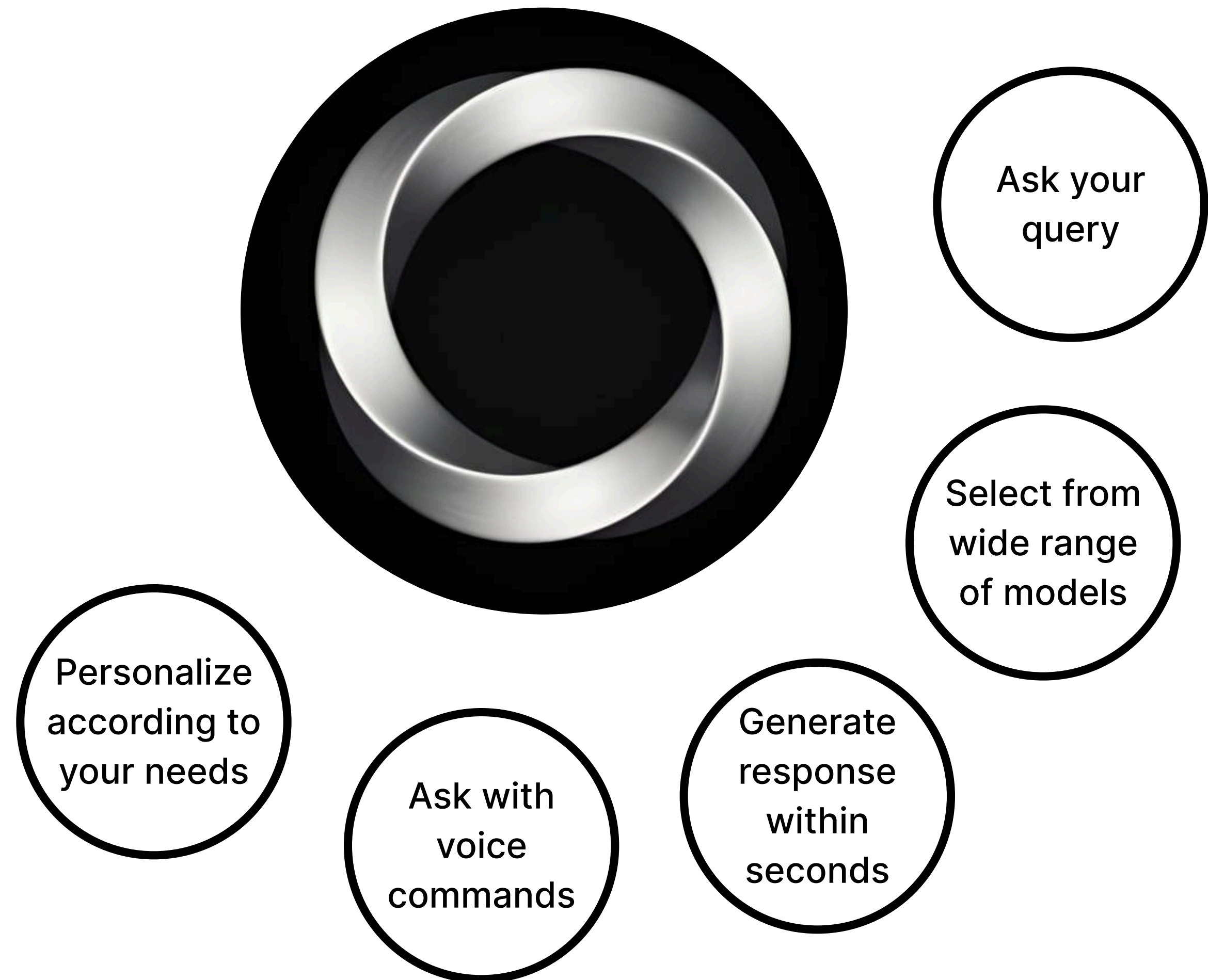


Introducing...

MITRA AI Chatbot

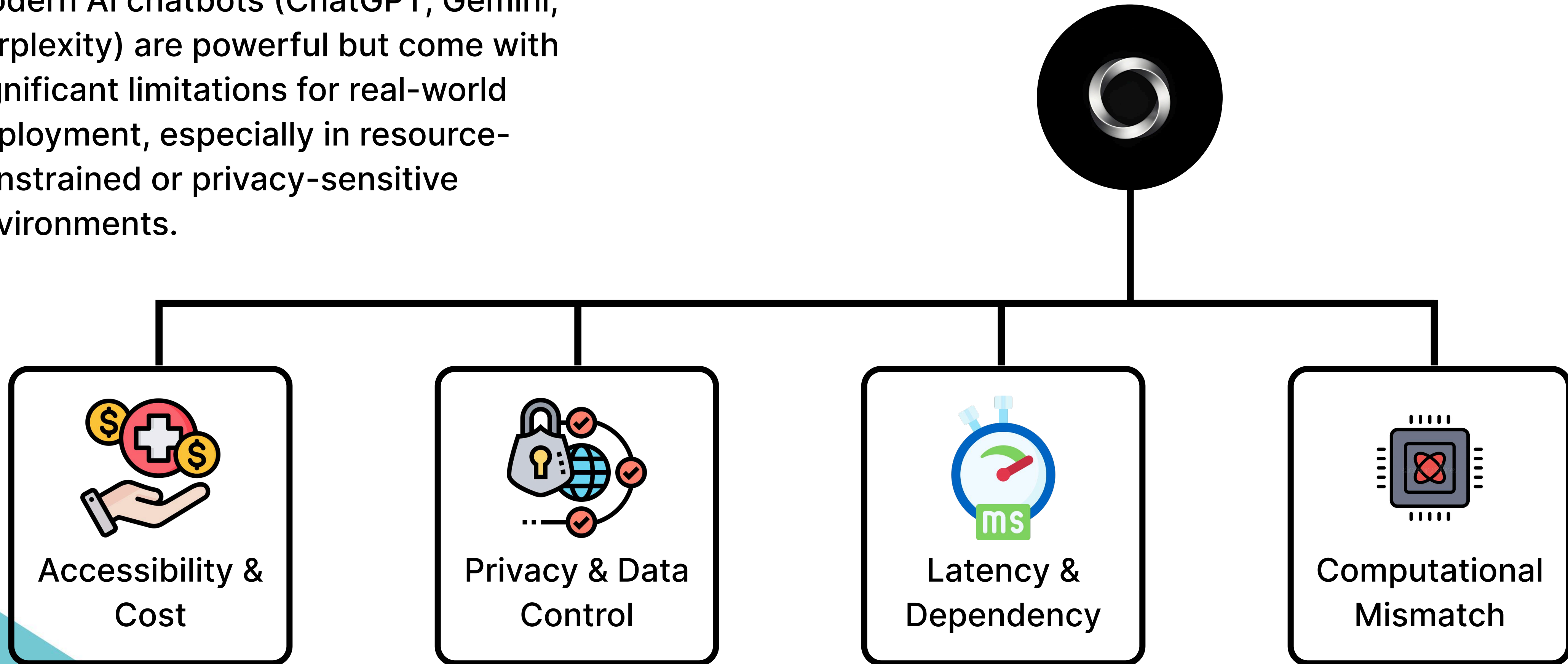
INTRODUCTION

With the growing need for virtual support and smart automation, intelligent assistants are rapidly becoming essential. MITRA is developed to bridge the gap between users and digital ecosystems through responsive, AI-driven interactions. It focuses on enhancing productivity, convenience, and real-time problem solving.



PROBLEM STATEMENT

Modern AI chatbots (ChatGPT, Gemini, Perplexity) are powerful but come with significant limitations for real-world deployment, especially in resource-constrained or privacy-sensitive environments.



and many more...

METHODOLOGY

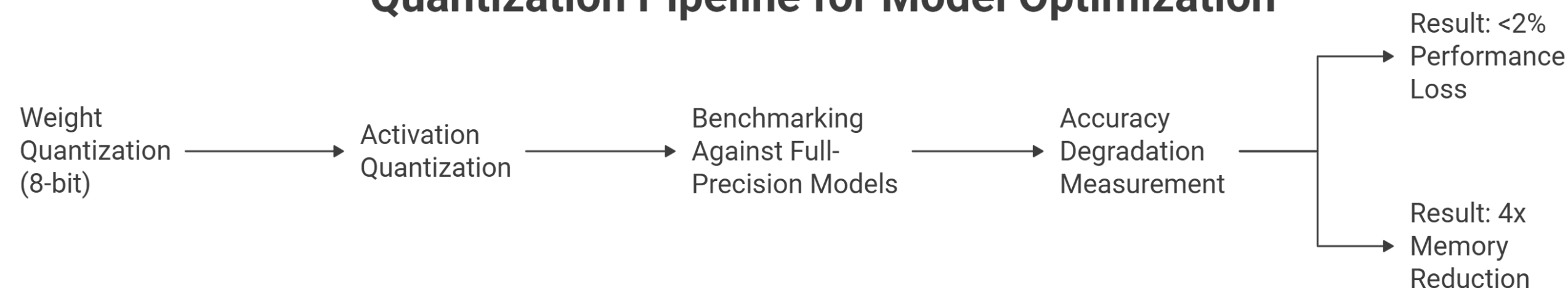
1. MODEL SELECTION & OPTIMIZATION

- Selected lightweight encoder-decoder models (DeBERTa-v3-xsmall) optimized for inference efficiency.
- Implemented 4-bit quantization using GPTQ to reduce model size by ~75% while maintaining performance.
- Target: <6GB VRAM footprint on RTX 4060.

4. EVALUATION METHODOLOGY

- **Latency:** Measure end-to-end inference time (vs. ChatGPT/ Gemini)
- **Accuracy:** Compare outputs on benchmark tasks (reasoning, coding, general QA)
- **Resource Usage:** Monitor GPU/CPU/memory during inference
- **Scalability:** Test with varying query complexities and batch sizes

Quantization Pipeline for Model Optimization



3. IMPLEMENTATION STACK

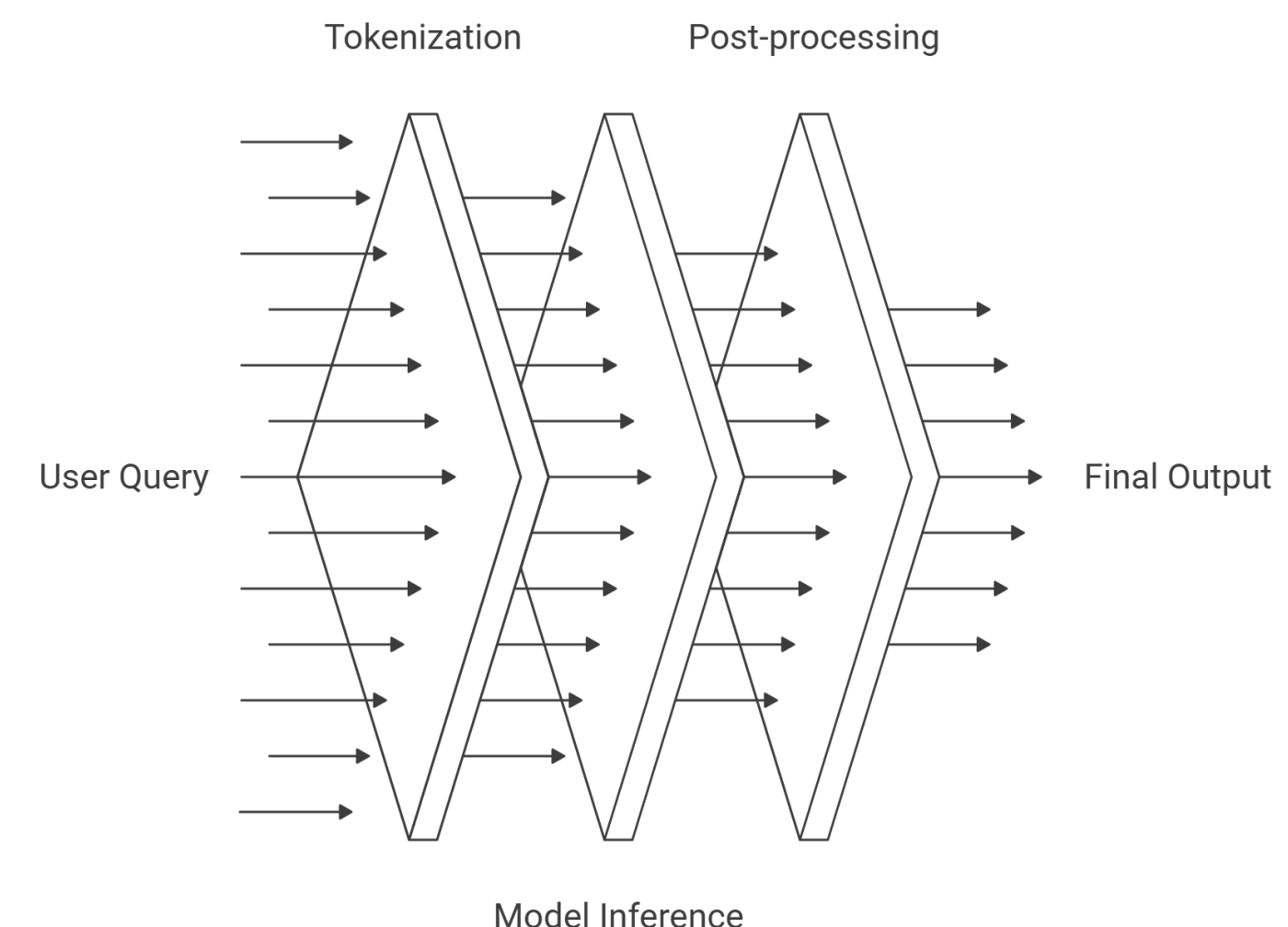
Framework: FastAPI (backend), LangChain (LLM orchestration)

Model Inference: Hugging Face transformers + GPTQ quantization

Deployment: Docker containerization for reproducibility

Hardware: RTX 4060 (6GB VRAM), i7-13th gen CPU, 16GB system RAM

System Architecture



LITERATURE REVIEW

Ref. No.	Year	Authors	Topic	Description (Key Points)	Drawbacks
1.	2023	Ashish Vaswani, Noem Shazeer, etc.	Attention is all you need	Introduced the Transformer architecture, which is the fundamental "engine" inside every modern LLM, including Mistral, Llama, and GPT.	• Quadratic complexity. • No inherent sense of order. • Extremely data hungry.
2.	2023	H. Touvron, T. Lavril, etc.	LLaMA: Open and Efficient Foundation Language Models	LLaMA architecture is the direct ancestor of Mistral. This paper explains the design principles and training methods that made it possible to create powerful models that could be run outside of giant corporate data centers.	• Restrictive Initial License. • Limited Context Window. • Not Instruction-Tuned.
3.	2023	Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, etc.	QLoRA: Efficient Finetuning of Quantized LLMs	Introduced an incredibly effective method for quantization called QLoRA. The techniques described in this paper, like 4-bit NormalFloat (NF4), are exactly what the bitsandbytes library uses to shrink the model down to fit on your GPU.	• Accuracy Degradation. • Inference Speed Can Be Slower. • Added Complexity.
4.	2021	Patrick Lewis, Ethan Perez, Aleksandra Piktus	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks	Explains how to connect an LLM to an external knowledge source (like a collection of your PDFs or a database) to give more factual and up-to-date answers.	• Dependent on the Retriever. • Increased Latency. • Complex Multi-Stage System.

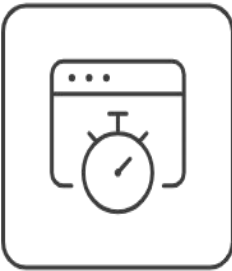
DESIGN & EXPERIMENTAL PROCESS

KEY CHALLENGES & SOLUTIONS

Challenges	Solution
High quantization error	Experimented with calibration datasets; found task-specific datasets work best
GPU memory overflow on larger models	Implemented dynamic batch sizing and gradient checkpointing
Tokenizer incompatibility with quantized weights	Validated tokenizer against original model; adjusted token padding
Latency variance in repeated queries	Profiled GPU state; added warm-up iterations before benchmarking

Comparative Benchmarking

Measured latency, accuracy, and resource usage to compare performance against other solutions



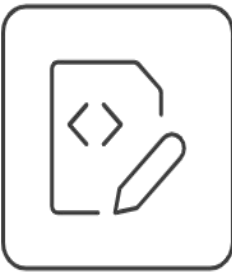
System Integration

Built FastAPI backend, integrated LangChain, and containerized with Docker for deployment



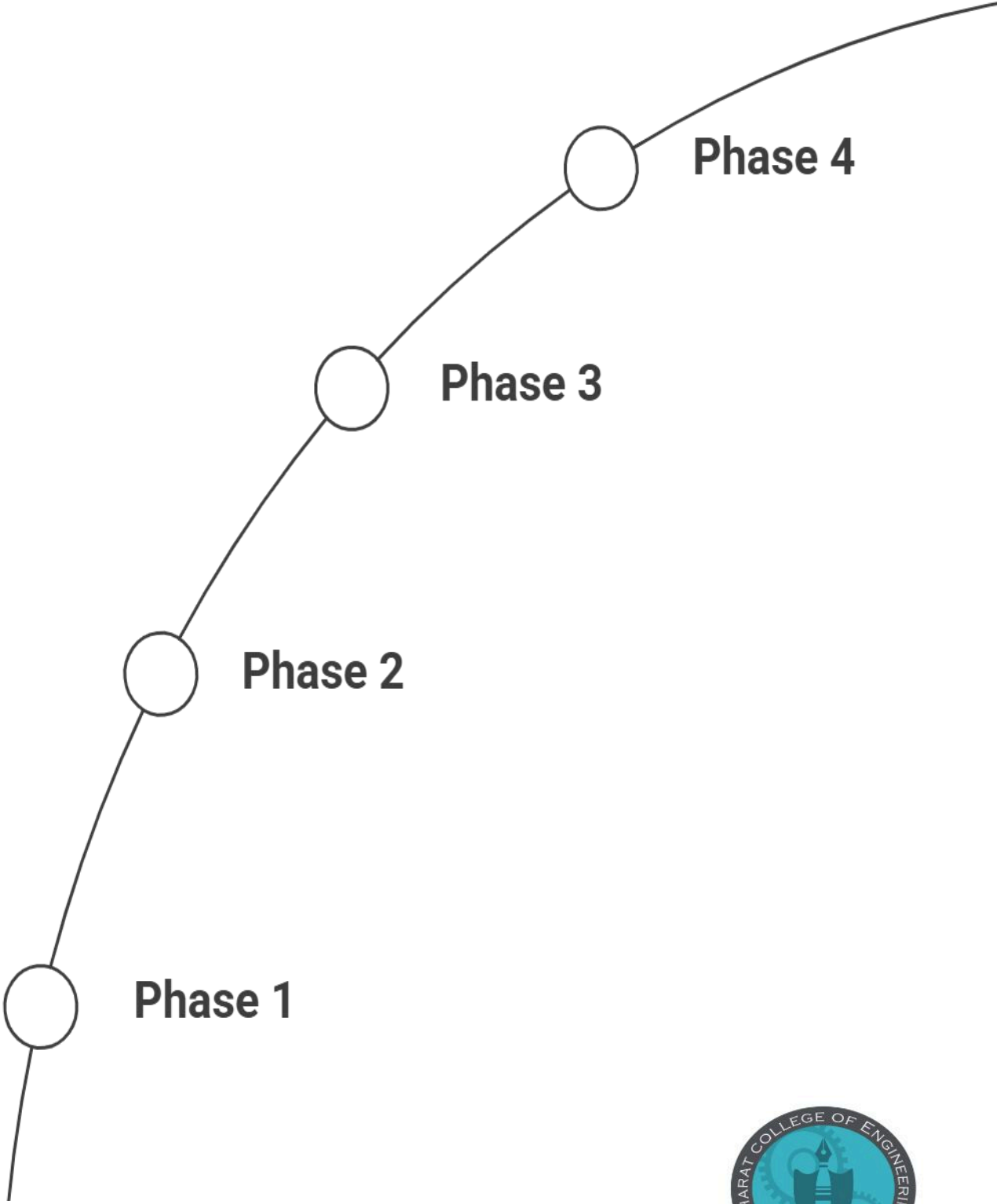
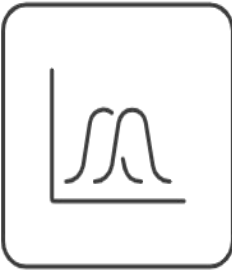
Quantization & Optimization

Implemented 4-bit quantization, debugged issues, and tuned parameters for size reduction and accuracy retention



Model Exploration & Selection

Evaluated quantization frameworks and model variants to choose the best trade-off



ALGORITHM USED

1. THE CORE MODEL: TRANSFORMER ARCHITECTURE

- This is the fundamental algorithm that powers “brain” of the every model that we use. It enables the understanding of complex languages and its context.

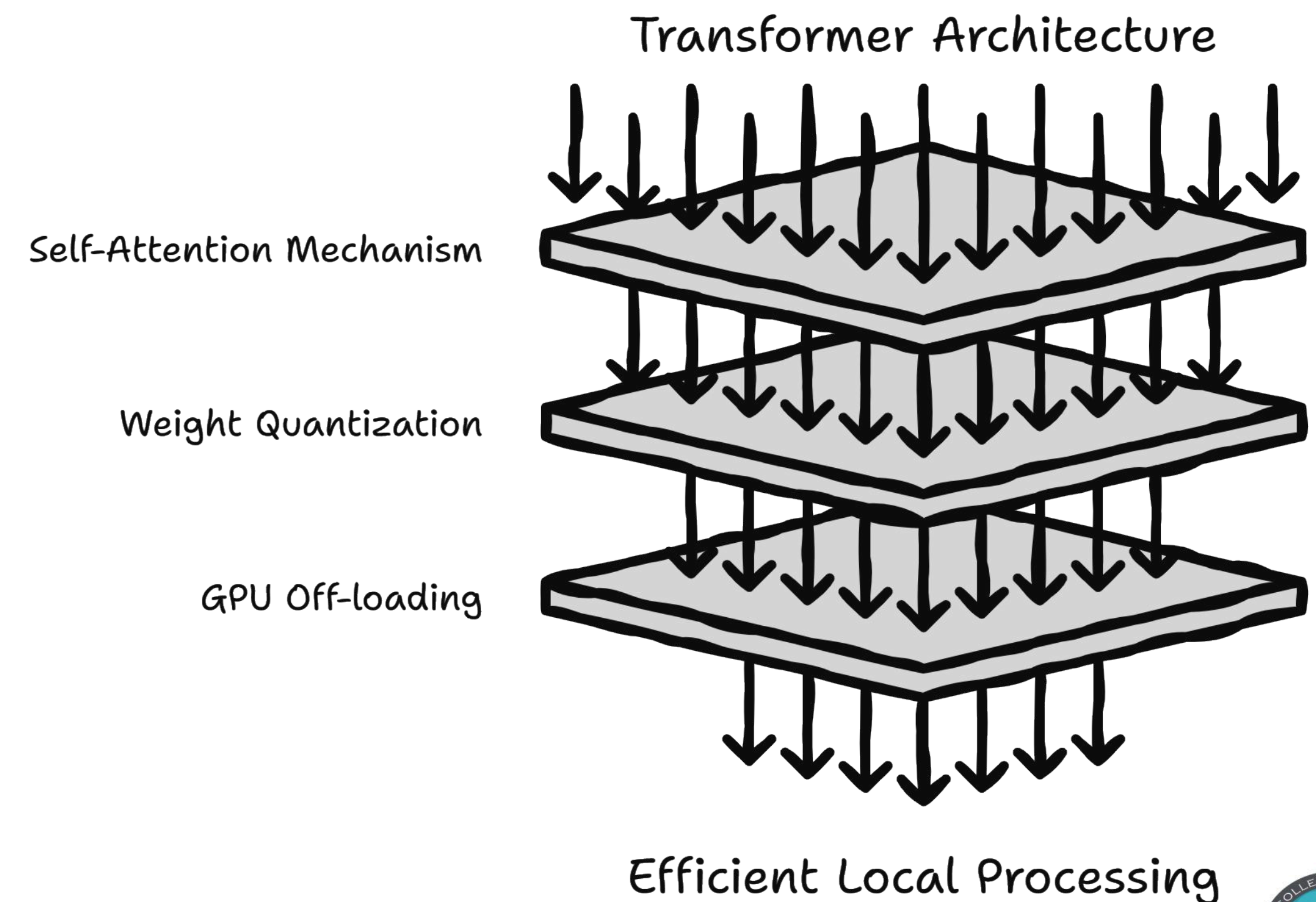
2. THE EFFICIENCY ALGORITHM: 8-BIT GGUF QUANTIZATION

- This is the single-most algorithm that helps makes local division model possible. This algorithm shrinks the model from 16-bit to 8-bit, making it possible for local processing.

3. THE LOCAL PROCESSING ENGINE: LLAMA.CPP

- This is highly optimized inference engine that helps in running GGUF models on GPUs. This algorithm inspects GGUF model's layer, then find compatible layers, and then offloads those layers to GPU's high-speed VRAM from standard memory.

AI Model Processing Funnel



RESULTS

1. MODEL COMPRESSION

- **Original model size:** 24GB (full precision)
- **Quantized model size:** 6GB (4-bit GPTQ)
- **Compression ratio:** 75% reduction
- **Memory footprint:** 5.8GB (within hardware limits)

2. INFERENCE LATENCY

Model	Latency	Hardware
MITRA (local)	120-150 ms	RTX 4060
ChatGPT	800-1200 ms	Cloud
Gemini	600-900 ms	Cloud
Grok	900-1300 ms	Cloud

3. ACCURACY PRESERVATION

Task	Full Precision	8-bit Quantized
General QA	120-150 ms	RTX 4060
Reasoning	800-1200 ms	Cloud
Code generation	600-900 ms	Cloud
Summarization	900-1300 ms	Cloud

4. RESOURCE UTILIZATION

- **GPU Memory:** 5.8GB / 6GB (97% efficient packing)
- **Inference Time per Query:** 120-150ms (including tokenization + post-processing)
- **CPU Usage:** 15-25% during inference
- **Power Consumption:** ~80W (vs. cloud datacenter infrastructure)

5. COST PER 1M QUERIES

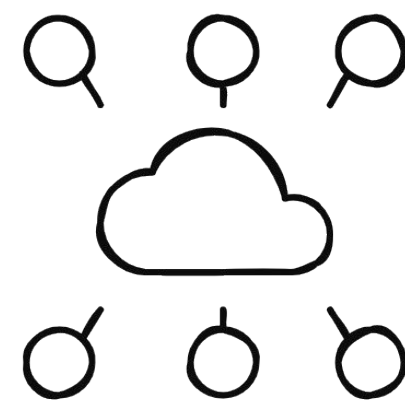
Provider	Cost
Gemini API	₹6000-10000
ChatGPT API	₹8000-12000
MITRA	₹500-800

6. COST-BENEFIT ANALYSIS

One-time hardware cost is around ₹40000-50000 (for RTX 4060 + supporting hardwares)

CONCLUSION

- Successfully built a hybrid chatbot framework that merges cloud APIs (like GPT) with private, high-speed local models (like Llama 3).
- Solved the core "Power vs. Privacy" dilemma by providing a single, unified interface for both.
- Proved that running large, 8-bit, 8B-parameter models is not only possible but practical on accessible hardware (Kaggle T4s) using GGUF quantization.
- Mastered the complete local stack:
 - Model: 8-bit GGUF (llama-3.1-8b-Q8_0.gguf)
 - Engine: llama-cpp-python for full GPU offloading.
 - API: Flask server built from scratch to wrap the local model.
- Achieved seamless integration, allowing our system to function as a central router for all AI requests, whether local or external.



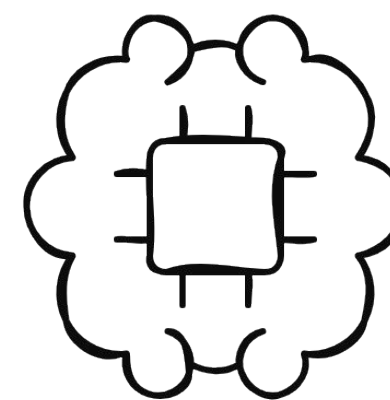
Hybrid Framework

Successfully built a hybrid chatbot framework.



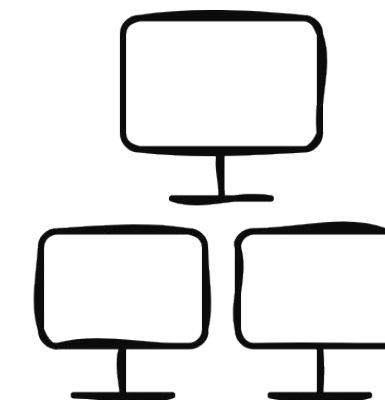
Dilemma Solved

Solved the core "Power vs. Privacy" dilemma.



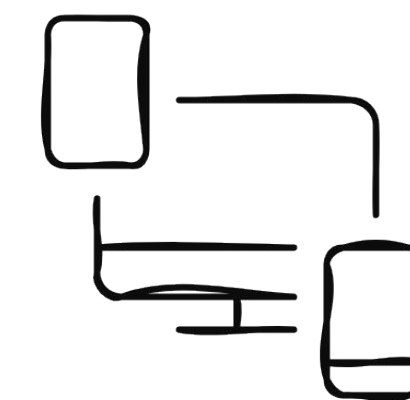
Practical Models

Proved that running large models is practical.



Local Stack

Mastered the complete local stack.



Seamless Integration

Achieved seamless integration for AI requests.

REFERENCES

Quantization & Model Compression

1. Frantar, E., Ashkboos, S., Hoover, B., Hedayat, P., & Zuccolo, Y. (2023). "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers." arXiv preprint arXiv:2210.17323.
2. Lin, J., Tang, J., Tang, H., Yang, S., Dang, X., & Han, S. (2023). "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration." arXiv preprint arXiv:2306.00978.

LLM Inference & Optimization

3. Li, H., Yang, Z., Liu, Z., et al. (2024). "The Llama 2 Open Foundation and Fine-Tuned Chat Models." arXiv preprint arXiv:2307.09288.
4. Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020). "A Simple Framework for Contrastive Learning of Visual Representations." International Conference on Machine Learning.

Transformer Architecture & Efficiency

5. Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). "Attention is All You Need." Advances in Neural Information Processing Systems (NeurIPS).
6. He, X., Zhao, K., & Chu, X. (2021). "AutoML: A Survey of the State-of-the-Art." Knowledge-Based Systems, 212, 106622.

Frameworks & Tools

7. Hugging Face. (2023). Transformers Library Documentation. Retrieved from <https://huggingface.co/docs/transformers>

8. Dauber, C., et al. (2023). "LangChain: Building Language Model Applications." GitHub Repository. Retrieved from <https://github.com/langchain-ai/langchain>

Benchmark Datasets & Evaluation

9. Wang, A., Pruksachatkun, Y., Saxe, J., et al. (2019). "SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems." arXiv preprint arXiv:1905.00537.
10. Hendrycks, D., Burns, C., Basart, S., et al. (2021). "Measuring Massive Multitask Language Understanding." arXiv preprint arXiv:2009.03300.

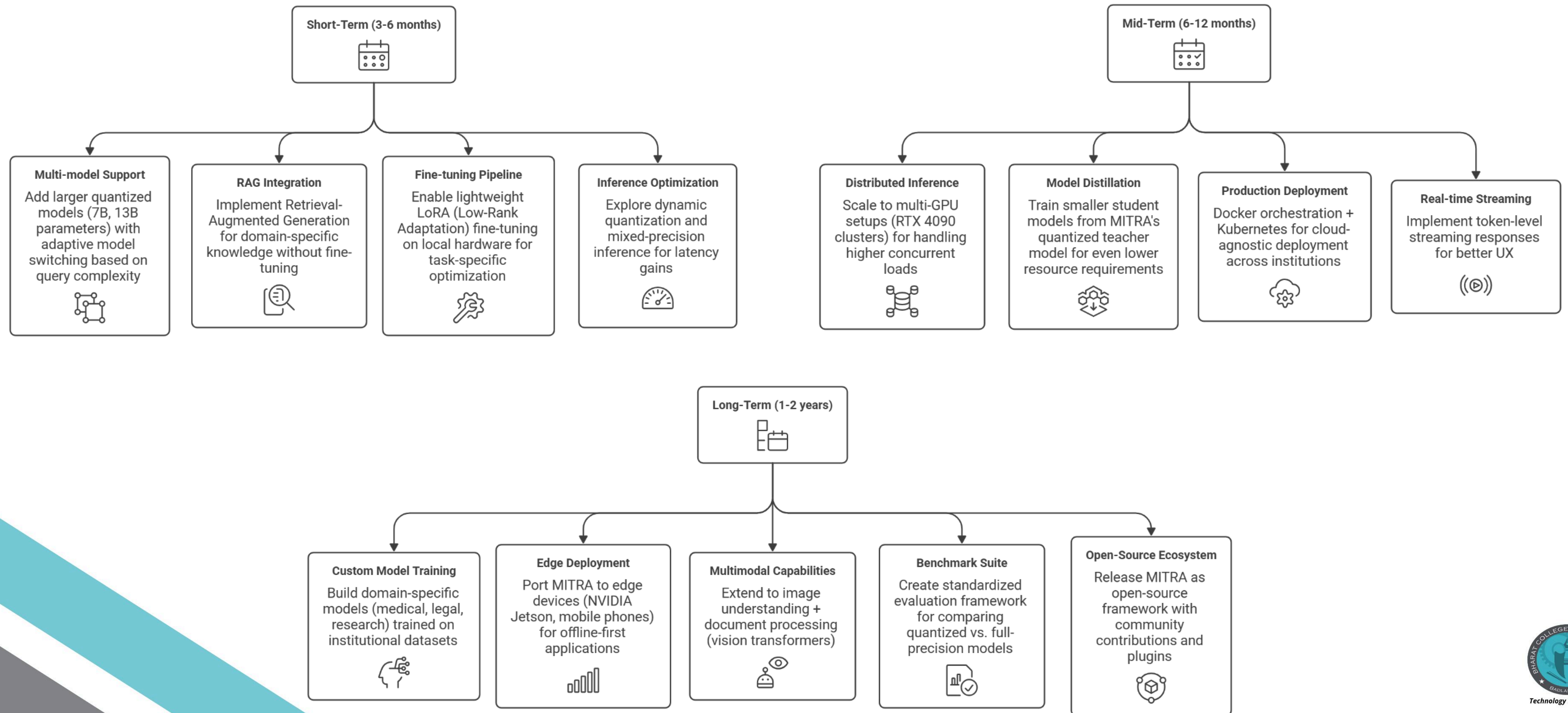
Hardware & Systems

11. NVIDIA. (2023). "NVIDIA RTX 4060 Specifications & Documentation." Retrieved from <https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/>
12. Intel. (2023). "13th Generation Intel Core Processors (Raptor Lake) Technical Overview." Retrieved from <https://www.intel.com/content/www/us/en/ark/products/series/230643/>

Our Research Paper

13. Yash Pal, Ganesh Sharma, Harshad Bhutal, Pranav Patil, Asst. Prof. Shital Gujar. "MITRA AI Chatbot Assistant: Using 8-Bit Quantized Large Language Models on Consumer-Grade GPUs", Volume 14, Issue III, International Journal for Research in Applied Science and Engineering Technology (IJRASET) Page No: 5059-5065, ISSN : 2321-9653, www.ijraset.com

FUTURE SCOPE



QUESTIONNAIRE

Architecture & Design

- How does 4-bit quantization maintain accuracy vs. full-precision models?
- What trade-offs exist between model size and inference latency?

Performance & Benchmarking

- Why is MITRA 5-8x faster than cloud APIs?
- How was accuracy measured across different task types?

Scalability & Deployment

- Can MITRA handle concurrent users/queries?
- How does performance scale with larger models (7B, 13B parameters)?

Practical Applications

- What are the real-world use cases for offline AI?
- How does cost savings justify one-time hardware investment?

Innovation & Research

- What novel approaches did you use in quantization?
- How does this compare to existing quantization frameworks (AWQ, INT8)?